Project Title

Resume–Job Matching & Talent-Marketplace Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Ramannagari Bhanu Priya (2520090171)
Daivapu Ananya (2520090011)

Team Number: 11

Section: 12

Under the guidance of

Ms. K. Chandusha

Assistant Professor, Department of CS&IT

(DEEMED TO BE UNIVERSITY)
BACHUPALLY
Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Resume–Job Matching & Talent-Marketplace Engine"** is a record of Bonafide work of **Ramannagari Bhanu Priya – 2520090171, Daivapu Ananya – 2520090011** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Ramannagari Bhanu Priya – 2520090171

Daivapu Ananya – 2520090011

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Resume–Job Matching & Talent-Marketplace Engine"** is a bonafide work done and submitted by **Ramannagari Bhanu Priya – 2520090171, Daivapu Ananya – 2520090011** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, Ms. K. Chandusha, Assistant Professor, Department of Computer Science and Information Technology, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Ramannagari Bhanu Priya – 2520090171

Daivapu Ananya – 2520090011

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	18
9.	References	19

1.ABSTRACT

The Resume–Job Matching & Talent-Marketplace Engine is an intelligent recruitment platform that connects job seekers and employers through automated résumé analysis and smart job recommendations. Manual résumé screening is slow, inconsistent and prone to bias, so the system extracts skills, education and experience from résumés and job postings and compares them to produce a transparent match score. The implementation is built around classical data structures and algorithms: a Trie stores the skill dictionary so multi-word skills are found in a single scan; hash sets hold every candidate's and job's skills so that overlap and gaps are computed with fast set intersection and difference; sparse term-frequency vectors with cosine similarity measure textual closeness; and merge sort together with a heap-based Top-K selection rank the candidates. A weighted score combines required-skill coverage (50%), preferred-skill coverage (15%), text similarity (20%) and experience fit (15%). Recruiters obtain a ranked shortlist, while job seekers obtain a skill-gap report with learning resources. On five sample résumés the best-fit candidate was ranked first with a score of 71.6%, and on 10,000 synthetic scores the full merge-sort ranking took about 23.1 ms while the heap-based Top-10 selection took about 1.2 ms. The work shows how fundamental DSA concepts can power a fast and explainable talent marketplace.

Team Name: Team 11 (Section 12)

Team Logo (if any): Not applicable

Team Members:

1. Ramannagari Bhanu Priya, 2520090171
2. Daivapu Ananya, 2520090011

2. INTRODUCTION

Hiring has moved almost entirely online. A single job opening can attract hundreds or thousands of applications, while job seekers struggle to find openings that truly fit their profile. Most screening is still done by reading résumés one at a time, or by keyword filters in traditional Applicant Tracking Systems. Both approaches are slow and inconsistent, and keyword filters miss synonyms and related skills, which leads to good candidates being overlooked.

This project builds a **Resume–Job Matching & Talent-Marketplace Engine** to address that gap. The engine converts unstructured résumés and job descriptions into structured profiles (skills, education, experience), computes a compatibility score for every candidate–job pair, ranks the results, and reports the skills a candidate is missing together with suggested learning resources. The aim is a single marketplace in which employers receive ranked, relevant candidate pools and job seekers receive better-matched opportunities and clear feedback.

The project is driven by Data Structures and Algorithms because matching must stay fast as the number of résumés grows. A **Trie** finds skills in text in one pass; **hash sets** give constant-time lookup, automatic de-duplication and quick intersection/difference between candidate and job skills; **sparse vectors with cosine similarity** compare the wording of documents; and **merge sort** and a **heap** produce the ranked shortlist in O(n log n) and O(n log k) time respectively.

The core engine is implemented as a Python prototype of four modules and tested on sample résumés and synthetic data. The web layer and database listed in the requirements (Flask/Django, React, MySQL/MongoDB) form the intended deployment environment for the engine. The rest of the report is organised as follows: Section 3 reviews similar systems, Section 4 states the problem and objectives, Section 5 lists the requirements, Section 6 presents the implementation, Section 7 discusses the results, and Section 8 concludes with future scope.

3. LITERATURE SURVEY

Several open-source projects and published models address résumé–job matching. The eight systems below were studied; their links are given in the References section.

[1] TAP (Test, Apply, Progress). A job-seeker and recruiter platform in which résumés are read with OCR and processed with NLP (spaCy) before being compared with job posts. Similarity is checked using cosine, Jaccard and Pearson scores to recommend jobs to candidates and candidates to recruiters. It focuses on recommendation quality and does not discuss the cost of ranking large pools.

[2] Match Made. A résumé screening dashboard that cleans text with spaCy, converts résumés and job descriptions into dense sentence-transformer embeddings, and combines semantic similarity (70%) with a skill-coverage score (30%). It is presented as a decision-support tool that still needs human oversight. Its dependence on an embedding model makes the score harder to explain than direct skill matching.

[3] SkillLadder. An IEEE-linked research project built with React, FastAPI and spaCy. It parses uploaded PDFs, scores them against job descriptions using vector similarity, performs skill-gap analysis and recommends jobs and learning resources, explaining why each job fits. It is close to our goal of combining matching with gap analysis, but it emphasises the web stack rather than data-structure design.

[4] Job.Query. Matches a résumé with a job description through a pipeline of extraction, cleaning, tokenising, embedding and cosine similarity so that HR can rank résumés by relevance. It also adds an applicant chatbot built on a LangChain SQL agent. Matching is a single similarity score, with no skill-level breakdown for the candidate.

[5] AI-Powered Resume Screening System (FUTURE_ML_03). A FastAPI and React application that extracts required skills from the job description dynamically with spaCy, reads PDF and text résumés, and mixes keyword intersection with TF-IDF cosine similarity to give a percentile match score. The idea of mixing set overlap with text similarity is similar to ours, but ranking and complexity are not analysed.

[6] spacy-job-recommendation (AventIQ-AI). A spaCy-based model that identifies skills through named-entity recognition, scores jobs with CountVectorizer and cosine similarity, and

adds graph-based relationship analysis and visualisation of the top matches. The model page reports about 85.6% accuracy. Graph analysis helps context, but results depend on the quality of entity extraction.

[7] ResumeMatcherLocalNLP. A React and Flask application where the user uploads a résumé, pastes a job description and receives a match score with suggestions for improvement, using cosine similarity and tokenisation with local NLP only. It serves a single job seeker at a time and has no recruiter-side ranking.

[8] Resume Matcher (srbhr). A widely used open-source tool that imitates an ATS: it parses a résumé and job description in Python, extracts key terms, measures vector similarity, and shows a match score with keyword suggestions. It is aimed at job seekers tailoring their résumés rather than at a two-sided marketplace.

Summary of the gap. Most existing work relies on NLP libraries and cosine similarity and serves either the recruiter or the job seeker. Few of them explain which data structures keep matching and ranking efficient, and few combine recruiter ranking with candidate skill-gap feedback. This project addresses that by building the engine on explicit data structures (Trie, hash sets, heap, merge sort) and by serving both sides of the marketplace.

4. Problem Statement and Objectives

Problem Statement: Manual résumé screening is slow, inconsistent and biased, and there is no fast, transparent, algorithm-driven way to match candidates with jobs at scale.

Problem Description: Recruiters receive more résumés than they can read and keyword filters overlook related skills, while job seekers do not know why they are rejected or what to learn next. A DSA-driven engine is needed to parse résumés, score each candidate against a job, and rank the results quickly.

Objectives:

Identify the Project's Purpose: The project was initiated to remove the slowness, inconsistency and bias of manual screening by building one marketplace in which employers get ranked, relevant candidate pools and job seekers get better-matched opportunities. The objectives are to:

- Parse résumés and job postings into structured profiles containing skills, education and experience.
- Store the skill vocabulary in a Trie and each profile's skills in hash sets for fast lookup, de-duplication and set operations.
- Compute a weighted compatibility score from required-skill coverage, preferred-skill coverage, text similarity (cosine) and experience fit.
- Rank candidates efficiently using merge sort and heap-based Top-K selection.
- Provide skill-gap analysis with learning recommendations for job seekers.
- Verify correctness and measure scalability on sample and synthetic data.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i5 (10th Gen) or higher
- **RAM:** 8 GB minimum (16 GB recommended)
- **Storage:** 256 GB SSD or higher
- **Graphics:** Integrated GPU (dedicated GPU optional for ML training)
- **Display:** 1920 × 1080 resolution
- **Connectivity:** Stable internet connection for cloud NLP APIs
- **Peripherals:** Standard keyboard and mouse

Software Requirements:

- **Operating System:** Windows 10/11 or Linux (Ubuntu 20.04+)
- **Programming Language:** Python 3.x
- **NLP / ML Libraries:** spaCy, NLTK, scikit-learn
- **Backend Framework:** Flask / Django (REST APIs)
- **Frontend:** HTML5, CSS3, JavaScript / React
- **Database:** MySQL / MongoDB
- **IDE / Tools:** VS Code, Jupyter Notebook, Git & GitHub

Note: the core engine in Section 6 uses only the Python standard library, so it runs without any of the optional packages above.

6. IMPLEMENTATION

The core engine is written in Python 3 as four modules. A small driver script (main.py) runs the whole pipeline on five sample résumés and one job posting (Python NLP Engineer); its output is shown in Section 7.

Module 1: Skill dictionary in a Trie (module1_skill_trie.py)

Each multi-word skill is stored token by token in a Trie whose children are hash maps. Scanning a document left to right, the longest matching phrase at each position is taken and added to a hash set, so duplicates disappear automatically. Cost is O(L) per position, where L is the length of the longest skill phrase.

```
"""Module 1 - Skill dictionary stored in a Trie (children kept in hash maps)."""

class TrieNode:
      slots   = ("children", "skill")
    def __init__(self):
        self.children = {}      # hash map: token -> TrieNode
        self.skill = None       # canonical skill name if a phrase ends here



class SkillTrie:
    """Multi-word skill phrases are stored token by token so that the
    longest phrase starting at any position is found in O(L)."""
    def __init__(self, skills):
        self.root = TrieNode()
        for s in skills:
            self.insert(s)

    def insert(self, phrase):
        node = self.root
        for tok in phrase.lower().split():
            node = node.children.setdefault(tok, TrieNode())
        node.skill = phrase.lower()

    def extract(self, text):
        """Return the SET of canonical skills found in text (hash set =
        automatic de-duplication)."""
        toks = [t.strip(".,;:()[]") for t in text.lower().replace("/", " ").split()]
        found, i = set(), 0
        while i < len(toks):
            node, j, last = self.root, i, None
            while j < len(toks) and toks[j] in node.children:
                node = node.children[toks[j]]
                j += 1
                if node.skill:
                    last = (node.skill, j)
            if last:                            # longest match wins
                found.add(last[0])
                i = last[1]
            else:
                i += 1
        return found



SKILLS = ["python", "java", "c++", "javascript", "react", "node.js", "flask",
          "django", "sql", "mysql", "mongodb", "html", "css", "git", "docker",
          "machine learning", "deep learning", "nlp", "spacy", "nltk",
```

```
          "scikit-learn", "pandas", "numpy", "data structures", "algorithms",
          "rest api", "aws", "linux", "tensorflow", "data analysis"]
```

Module 2: Résumé and job parser (module2_resume_parser.py)

Normalises text (for example ML to machine learning), then extracts the name, e-mail, highest degree, years of experience and the skill set. A job posting is parsed into required and preferred skill sets and a minimum experience.

```
"""Module 2 - Resume / job-description parser (NLP-style extraction)."""
import re
from module1_skill_trie import SkillTrie, SKILLS

# 'ml' / 'dsa' are expanded before trie lookup
EXPAND = {r"\bml\b": "machine learning", r"\bdsa\b": "data structures algorithms",
          r"\breactjs\b": "react", r"\bjs\b": "javascript"}
TRIE = SkillTrie(SKILLS)

DEGREES = {"b.tech": 3, "btech": 3, "m.tech": 4, "mtech": 4, "mba": 4,
           "b.sc": 2, "m.sc": 4, "phd": 5}

def normalise(text):
    t = text.lower()
    for pat, rep in EXPAND.items():
        t = re.sub(pat, rep, t)
    return t

def parse_resume(raw):
    text = normalise(raw)
    lines = [l.strip() for l in raw.strip().splitlines() if l.strip()]
    email = re.search(r"[\w.+-]+@[\w-]+\.[\w.]+", raw)
    years = re.findall(r"(\d+(?:\.\d+)?)\+?\s*years?", text)
    degree = max((d for d in DEGREES if d in text),
                 key=lambda d: DEGREES[d], default="none")
    return {"name": lines[0],
            "email": email.group(0) if email else "-",
            "education": degree.upper(),
            "experience": max(map(float, years), default=0.0),
            "skills": TRIE.extract(text),          # hash set
            "tokens": re.findall(r"[a-z+#.]+", text)}

def parse job(title, required, preferred, min exp, description):
    return {"title": title, "required": TRIE.extract(normalise(required)),
            "preferred": TRIE.extract(normalise(preferred)),
            "min exp": min exp,
            "tokens": re.findall(r"[a-z+#.]+", normalise(description))}
```

Module 3: Matching engine (module3_matcher.py)

Required and preferred coverage are computed by hash-set intersection. Text similarity is the cosine of sparse term-frequency vectors stored as dictionaries. The final score is 50% required + 15% preferred + 20% text similarity + 15% experience fit.

```
"""Module 3 - Matching engine: set intersection + cosine similarity."""
import math
from collections import Counter

STOP = {"the", "and", "a", "an", "of", "in", "to", "with", "for", "is", "on",
        "we", "are", "at", "as", "by", "or", "has", "have", "using", "built"}

def tf_vector(tokens):
    return Counter(t for t in tokens if t not in STOP and len(t) > 1)
```

```
def cosine(a, b):
    """cos(A,B) = A.B / (|A||B|) on sparse TF vectors (dict). O(min(|A|,|B|))"""
    if len(a) > len(b):
        a, b = b, a
    dot = sum(v * b.get(k, 0) for k, v in a.items())
    na = math.sqrt(sum(v * v for v in a.values()))
    nb = math.sqrt(sum(v * v for v in b.values()))
    return dot / (na * nb) if na and nb else 0.0


def match score(resume, job):
    req, pref = job["required"], job["preferred"]
    matched req = resume["skills"] & req                # O(min(n,m)) hash lookups
    req_cov  = len(matched_req) / len(req) if req else 1.0
    pref_cov = len(resume["skills"] & pref) / len(pref) if pref else 1.0
    text sim = cosine(tf vector(resume["tokens"]), tf vector(job["tokens"]))
    need = job["min exp"]
    exp fit = min(resume["experience"] / need, 1.0) if need else 1.0
    score = 0.50 * req_cov + 0.15 * pref_cov + 0.20 * text_sim + 0.15 * exp_fit
    return round(score * 100, 1), {"required": round(req_cov * 100, 1),
                                   "preferred": round(pref_cov * 100, 1),
                                   "text": round(text_sim * 100, 1),
                                   "experience": round(exp_fit * 100, 1)}
```

Module 4: Ranking and skill-gap analysis (module4_ranking_gap.py)

A stable merge sort (O(n log n)) produces the full ranked list; a size-k min-heap gives the Top-K in O(n log k). The skill gap is the set difference between job skills and candidate skills, each mapped to a learning resource.

```
"""Module 4 - Ranking (merge sort + max-heap Top-K) and skill-gap analysis."""
import heapq


def merge_sort(items, key):
    """Stable O(n log n) descending merge sort."""
    if len(items) <= 1:
        return items
    mid = len(items) // 2
    L, R = merge sort(items[:mid], key), merge sort(items[mid:], key)
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if key(L[i]) >= key(R[j]):
            out.append(L[i]); i += 1
        else:
            out.append(R[j]); j += 1
    return out + L[i:] + R[j:]


def top k(items, k, key):
    """Max-heap selection: O(n log k) - best for large candidate pools."""
    heap = []                                    # min-heap of size k
    for idx, it in enumerate(items):
        entry = (key(it), -idx, it)              # idx breaks ties -> stable
        if len(heap) < k:
            heapq.heappush(heap, entry)
        else:
            heapq.heappushpop(heap, entry)       # drop the current minimum
    return [t[2] for t in sorted(heap, reverse=True)]


LEARNING = {"docker": "Docker Mastery (Docker Docs / Udemy)",
            "aws": "AWS Cloud Practitioner Essentials (AWS Skill Builder)",
            "react": "React - The Official Tutorial (react.dev)",
            "sql": "SQL for Data Science (Coursera)",
            "django": "Django Official Tutorial (djangoproject.com)",
            "nlp": "NLP Specialization (DeepLearning.AI)",
            "machine learning": "Machine Learning (Andrew Ng, Coursera)",
            "git": "Pro Git book (git-scm.com)",
            "flask": "Flask Quickstart (flask.palletsprojects.com)",
            "mongodb": "MongoDB University M001"}
```

```
def skill_gap(resume, job):
    missing_req = sorted(job["required"] - resume["skills"])       # set difference
    missing_pref = sorted(job["preferred"] - resume["skills"])
    plan = {s: LEARNING.get(s, "Search official docs / NPTEL course")
            for s in missing_req + missing_pref}
    return missing_req, missing_pref, plan
```

7. RESULTS & DISCUSSION

The engine was tested on five sample résumés against one job posting, "Python NLP Engineer" (required: Python, NLP, machine learning, SQL, Git; preferred: spaCy, scikit-learn, Docker, Flask, AWS; minimum experience 2 years). The terminal output of main.py is shown below, with a short explanation under each figure.

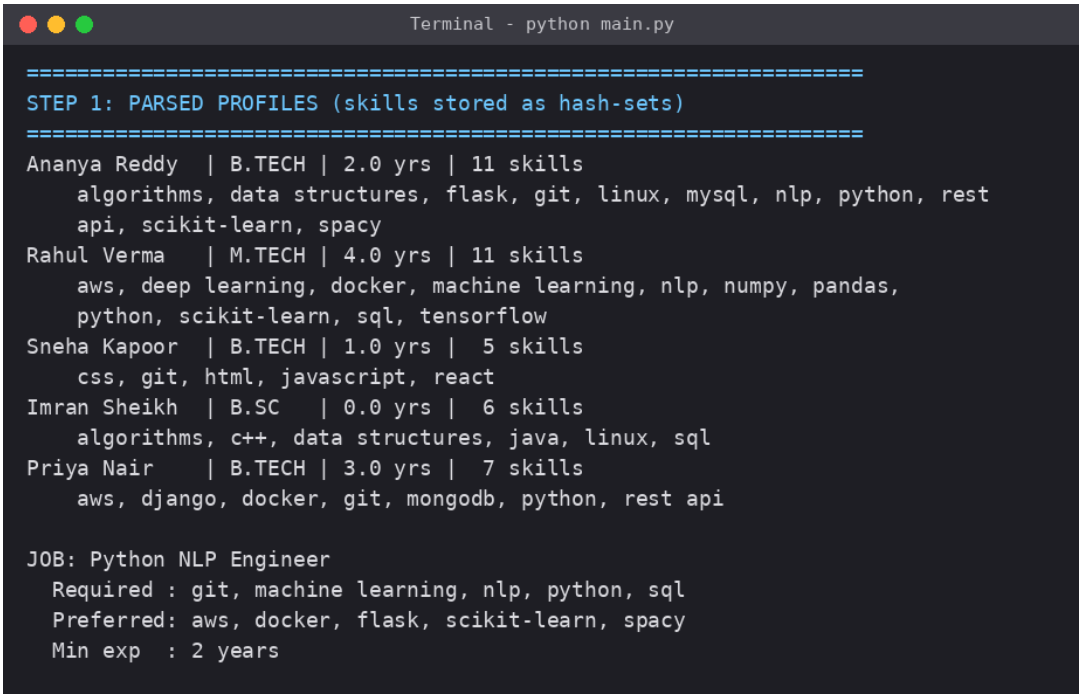


Figure 1: Parsed résumé profiles and job requirements

Each résumé is converted into education, years of experience and a de-duplicated skill set by the Trie-based extractor. The job posting is parsed into required and preferred skill sets.

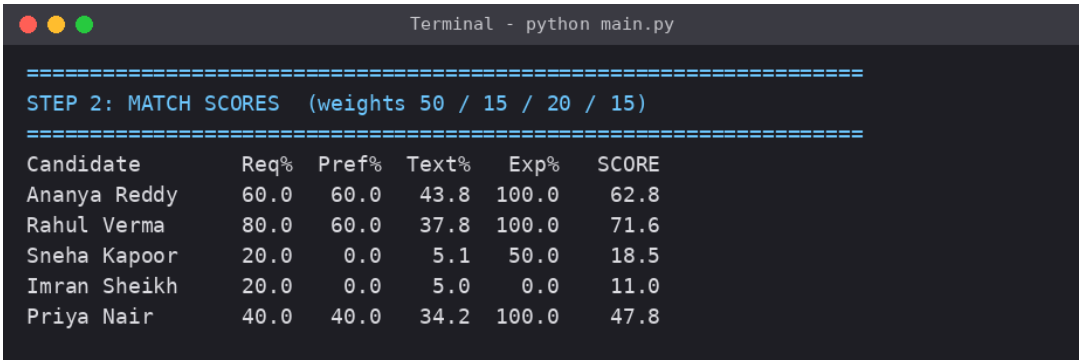


Figure 2: Component-wise and final match scores

Rahul Verma scores highest (71.6%) because he covers four of the five required skills, whereas Imran Sheikh scores lowest (11.0%) with one required skill and no experience.

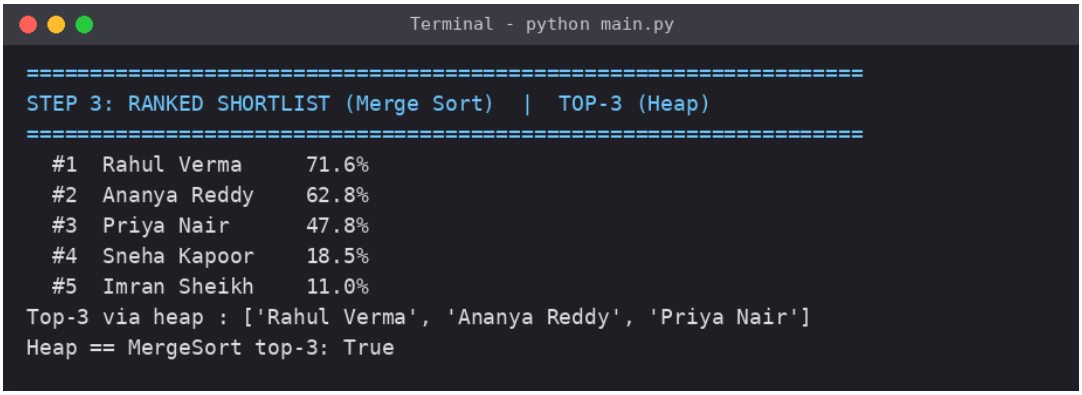


Figure 3: Ranked shortlist (merge sort) and Top-3 (heap)

Merge sort produces the complete ranking and the heap returns the Top-3 directly. Both give the same candidates in the same order, which confirms the two methods agree.

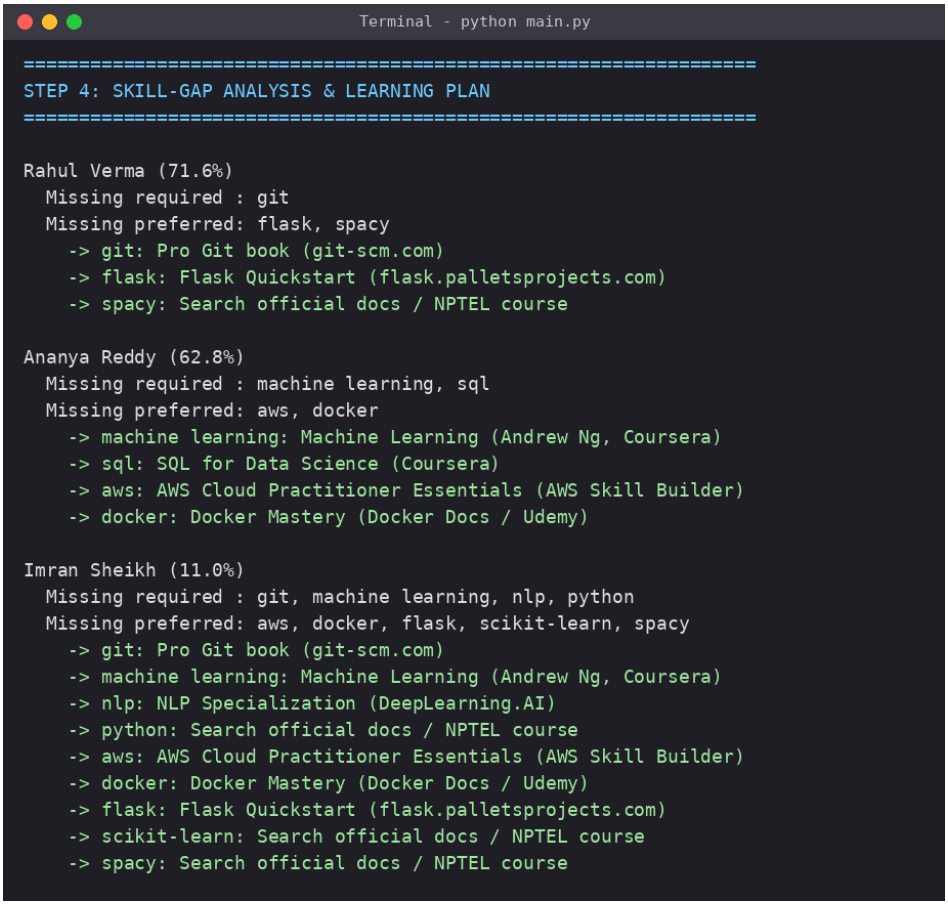


Figure 4: Skill-gap analysis and learning plan

For each candidate the engine lists the missing required and preferred skills and maps each to a learning resource. Skills without an entry in the resource table get a generic suggestion.

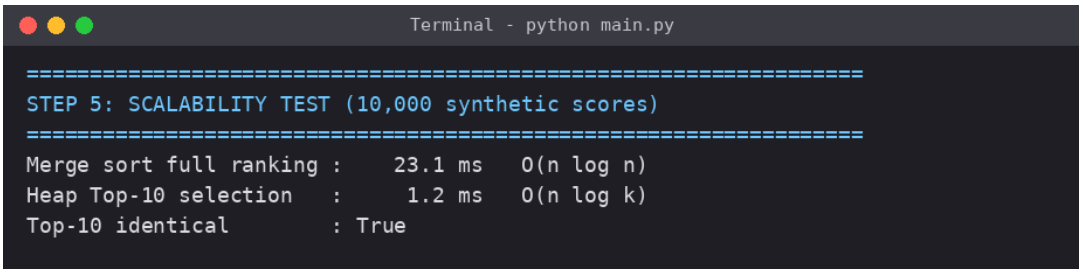


Figure 5: Scalability test on 10,000 synthetic scores

Heap-based Top-10 selection took about 1.2 ms against about 23.1 ms for sorting all scores, and both returned the same Top-10. Timings vary slightly from run to run.

Discussion. The ranking follows what one would expect from the skill sets: candidates covering more of the required skills rank higher, and experience and text similarity adjust the order. The set-based design makes the skill comparison transparent, since the exact matched and missing skills can be shown to the recruiter and the candidate. For large pools, the heap is preferable when only the best few candidates are needed, and merge sort when the full order is needed.

Limitations. Skill extraction is dictionary-based, so skills missing from the dictionary or written differently are not recognised; for example, “PostgreSQL” in Priya Nair’s résumé was not mapped to SQL, which lowered her required-skill coverage. The weights (50/15/20/15) are chosen by us and have not been tuned against hiring outcomes, and the test used only five hand-written résumés, so these results show that the pipeline works but do not measure accuracy against real recruiter decisions.

8. CONCLUSION & FUTURE SCOPE

The project set out to build a résumé–job matching engine on explicit data structures and algorithms. The prototype meets its main objectives: résumés and job postings are parsed into structured profiles; skills are stored in a Trie and hash sets; a weighted score is computed from skill overlap, cosine similarity and experience; candidates are ranked with merge sort and a heap; and a skill-gap report with learning suggestions is produced. On sample data the ranking was sensible, and the heap-based Top-K selection was clearly faster than a full sort. The web interface and database from the planned stack are not part of this prototype, and accuracy has not yet been validated on real hiring data.

Future Scope

Near term

- Add synonym and alias handling (for example PostgreSQL to SQL) and a larger skill dictionary.
- Read PDF and DOCX résumés directly and use spaCy named-entity recognition for extraction.
- Tune the scoring weights and report precision@k against recruiter-labelled data.

Medium term

- Wrap the engine in Flask/Django REST APIs with a React front end and a MySQL/MongoDB store.
- Add recruiter filters, saved searches and a candidate dashboard showing match breakdowns.
- Add embedding or TF-IDF based semantic matching alongside the skill-based score.

Long term

- Model skills as a graph so that related skills receive partial credit.
- Audit the scoring for bias and add explainability reports for fair hiring.
- Learn from recruiter feedback and scale the engine to cloud deployment with large candidate pools.

9. REFERENCES

1. TAP (Test, Apply, Progress) – résumé parsing and job recommendation. https://github.com/p1utoze/TAP

2. Match Made – résumé and job matching system. https://github.com/sonuj04/Match_Made

3. SkillLadder – AI-powered career development and job recommendation platform. https://github.com/linga-1221/skillladder

4. Job.Query – job and résumé matching with applicant chatbot. https://github.com/asfkfjhe/Job.Query

5. AI-Powered Resume Screening System (FUTURE_ML_03). https://github.com/Ayon-coder/FUTURE_ML_03

6. spacy-job-recommendation (AventIQ-AI). https://www.promptlayer.com/models/spacy-job-recommendation

7. ResumeMatcherLocalNLP. https://github.com/twittt/ResumeMatcherLocalNLP

8. Resume Matcher (srbhr). https://github.com/srbhr/resume-matcher

9. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

10. Python documentation – heapq (heap queue algorithm). https://docs.python.org/3/library/heapq.html